**本节内容：**

- Flex布局
- 过渡动画
- 2d转换

# 一、Flex布局

## 1.1 基本概念

### 1. 什么是 Flexbox?

- Flexbox（弹性盒子布局）是 CSS 中的一种布局模型，用于实现灵活的、响应式的布局。
- 它可以轻松地对齐和分配空间，尤其适合处理动态内容或未知大小的元素。

### 2. Flexbox 的主要用途

- 创建水平或垂直排列的布局。
- 实现居中对齐（水平、垂直）。
- 动态调整子元素的大小以适应容器。
- 构建导航栏、卡片布局、网格系统等。

### 3. Flexbox 的基本结构

- `容器` **（Flex Container）**：设置 `display: flex` 的元素。
- `子项` **（Flex Items）**：容器内的直接子元素。

## 1.2 容器属性

### (1) `display: flex`

- 将一个元素设置为 Flex 容器。
- 默认情况下，子元素会水平排列。

**案例：简单的水平排列**

```
<div class="container">
  <div class="item">Item 1</div>
  <div class="item">Item 2</div>
  <div class="item">Item 3</div>
</div>
```

```
.container {
  display: flex; /* 设置为 Flex 容器 */
}
.item {
  width: 100px;
  height: 100px;
  background-color: lightblue;
  margin: 5px;
}
```

**运行效果**：

- 子元素水平排列，均匀分布在容器内。

### 1.2.1 定义主轴方向

`flex-direction`

- 定义主轴的方向（即子元素的排列方向）。
- 可选值：
  - `row`： 默认,子元素沿水平方向从左到右排列。
  - `row-reverse`:子元素沿水平方向从右到左排列。
  - `column`:子元素沿垂直方向从上到下排列。
  - `column-reverse`:子元素沿垂直方向从下到上排列。

**案例：垂直排列**

```
.container {
  display: flex;
  flex-direction: column; /* 垂直排列 */
}
```

**运行效果**：

- 子元素从上到下垂直排列。

### 1.2.2 主轴对齐方式

`justify-content`

- 定义主轴上的对齐方式。
- 可选值：
  - `flex-start`:默认, 起点对齐
  - `flex-end`:子元素向主轴的终点对齐。
  - `center`:子元素在主轴上居中对齐。

- `space-between`:子元素之间的间距相等，首尾子元素分别贴紧主轴的起点和终点。
- `space-around`:每个子元素周围的间距相等。
- `space-evenly`:每个子元素之间的间距以及子元素与容器边缘的间距完全相等。

**案例：水平居中对齐**

```
.container {
  display: flex;
  justify-content: center; /* 水平居中 */
}
```

**运行效果**:

- 子元素在容器内水平居中对齐。

### 1.2.3 副轴对齐方式

`align-items`

- 定义交叉轴上的对齐方式。
- 可选值:
  - `stretch`： 默认，子元素在交叉轴方向上拉伸以填满容器。
  - `flex-start`： 子元素在交叉轴的起点对齐。
  - `flex-end`： 子元素在交叉轴的终点对齐。
  - `center`： 子元素在交叉轴上居中对齐。
  - `baseline`： 子元素在交叉轴上按基线对齐。

**案例：垂直居中对齐**

```
.container {
  display: flex;
  align-items: center; /* 垂直居中 */
  height: 200px; /* 设置容器高度 */
}
```

**运行效果**:

- 子元素在容器内垂直居中对齐。

### 1.2.4 子元素允许换行

`flex-wrap`

- 定义子元素是否换行。
- 可选值：`nowrap`（默认）、`wrap`、`wrap-reverse`。

**案例：子元素换行**

```
.container {
  display: flex;
  flex-wrap: wrap; /* 允许换行 */
}
.item {
  width: 150px; /* 子元素宽度大于容器宽度时会换行 */
}
```

**运行效果**：

- 当子元素的总宽度超过容器宽度时，自动换行。

### 1.2.5 子项行列间距

`gap`

- 控制子项之间的间距
- `gap: 行间距 列间距;`

**单个值**：同时设置行间距和列间距。

```
gap: 20px; /* 行间距和列间距都为20px */
```

**两个值**：第一个值设置行间距，第二个值设置列间距。

```
gap: 10px 30px; /* 行间距10px，列间距30px */
```

## 1.3 子项属性

### 1.3.1 子项拓展比例

`flex-grow`

- 定义子元素在主轴方向上的扩展比例。
- 默认值为 `0`，表示不扩展。
- **作用**：当容器的主轴上有剩余空间时，`flex-grow` 决定每个子元素占据多少剩余空间。

**案例：子元素按比例扩展**

```
.item:nth-child(1) {
  flex-grow: 1; /* 占 1 份 */
}
.item:nth-child(2) {
  flex-grow: 2; /* 占 2 份 */
}
```

### 1.3.2 子项是否收缩

`flex-shrink`

- 定义子元素在主轴方向上位置不足的时候是否收缩。
- 默认值为 `1`，表示允许收缩。

**案例：防止子元素收缩**

```
.item {
  flex-shrink: 0; /* 不允许收缩 */
}
```

**运行效果**：

- 子元素不会因为容器宽度不足而缩小。

### 1.3.3 初始子项长度

`flex-basis`

- 规定子项目的初始长度。

**案例：将第三个弹性项目的初始长度设置为 30 像素**

```
.flex-container {
  display: flex;
}

.flex-container > div {
  background-color: DodgerBlue;
  flex-basis: 30px;
}
```

### 1.3.4 复合属性flex

`flex`

- `flex` 属性是 flex-grow、flex-shrink 和 flex-basis 属性的简写属性。

### 案例：使第三个弹性项目可增长（1），不可收缩（0），且初始长度为 30 像素

```
.flex-container {
  display: flex;
}

.flex-container > div {
  background-color: DodgerBlue;
  flex: 1 0 30px;
}
```

## 1.4 实际应用

---

### 1.4.1 案例1：水平垂直居中

**需求**：让一个元素在容器内水平和垂直居中。

#### HTML 结构

```
<div class="container">
  <div class="box">box</div>
</div>
```

#### CSS 样式

```
.container {
  display: flex;
  justify-content: center; /* 水平居中 */
  align-items: center; /* 垂直居中 */
  height: 300px; /* 设置容器高度 */
  border: 1px solid #ccc;
}
.box {
  width: 100px;
  height: 100px;
  background-color: lightcoral;
}
```

**运行效果**：

- `.box` 在容器内水平和垂直居中。

### 1.4.2 案例2：导航栏布局

**需求**：创建一个水平排列的导航栏，左侧有 logo，右侧有菜单项。

#### HTML 结构

```html
<div class="navbar">
  <div class="logo">Logo</div>
  <div class="menu">
    <div class="menu-item">首页</div>
    <div class="menu-item">产品</div>
    <div class="menu-item">联系我们</div>
  </div>
</div>
```

### CSS 样式

```css
.navbar {
  display: flex;
  justify-content: space-between; /* 左右分布 */
  align-items: center; /* 垂直居中 */
  padding: 10px;
  background-color: #f0f0f0;
}
.menu {
  display: flex;
  gap: 10px; /* 设置菜单项之间的间距 */
}
.menu-item {
  padding: 5px 10px;
  background-color: lightblue;
  cursor: pointer;
}
```

# 二、过渡动画

## 2.1 基本概念

### 1. 什么是 CSS 过渡动画?

- CSS 过渡动画是一种用于在一定时间内平滑地改变元素样式的机制。
- 它可以让元素的样式变化（如颜色、大小、位置等）更加自然，而不是瞬间完成。

### 2. CSS 过渡动画的主要用途

- 创建按钮悬停效果（如颜色变化、放大缩小）。
- 实现简单的动画效果（如淡入淡出、滑动）。
- 增强页面的交互性和用户体验。

## 2.2 `transition` 的写法

```css
transition: property duration timing-function delay;
```

- `property`：需要过渡的 CSS 属性（如 `color`、`width`、`transform` 等）。
- `duration`：过渡持续时间（如 `0.5s` 表示 0.5 秒）。
- `timing-function`：过渡的速度曲线（如 `ease`、`linear`、`ease-in-out`）。
- `delay`：过渡延迟时间（可选，默认为 `0s`）。

## 2.3 详细参数

### 2.3.1 1. `property`（过渡属性）

- 指定需要过渡的 CSS 属性。
- 可以是具体的属性（如 `background-color`、`width`），也可以是 `all`（表示所有属性）。
- 示例：

```
transition: background-color 0.5s ease;
transition: all 0.5s ease;
```

### 2.3.2 2. `duration`（持续时间）

- 指定过渡的持续时间。
- 单位可以是秒（`s`）或毫秒（`ms`）。
- 示例：

```
transition: background-color 0.5s ease; /* 0.5 秒 */
transition: background-color 500ms ease; /* 500 毫秒 */
```

### 2.3.3 3. `timing-function`（速度曲线）

- 指定过渡的速度曲线。
- 常见值：
  - `ease`（默认值）：慢速开始和结束，中间加速。
  - `linear`：匀速变化。
  - `ease-in`：慢速开始，然后加速。
  - `ease-out`：快速开始，然后减速。
  - `ease-in-out`：慢速开始和结束，中间加速。
  - `cubic-bezier(n,n,n,n)`：自定义贝塞尔曲线。
    - 自定义贝塞尔曲线工具：https://cubic-bezier.tupulin.com/
- 示例：

```
transition: background-color 0.5s ease-in-out;
transition: background-color 0.5s cubic-bezier(0.1, 0.7, 1.0, 0.1);
```

### 2.3.4 4. `delay` （延迟时间）

- 指定过渡开始前的延迟时间。
- 单位可以是秒（`s`）或毫秒（`ms`）。
- 示例：

```
transition: background-color 0.5s ease 1s; /* 延迟 1 秒后开始 */
transition: background-color 0.5s ease 500ms; /* 延迟 500 毫秒后开始 */
```

## 2.4 核心用法

### 2.4.1 1. 单属性过渡

- 仅对某个特定的 CSS 属性应用过渡效果。
- 示例：

```
.box {
  width: 100px;
  height: 100px;
  background-color: lightblue;
  transition: background-color 0.5s ease;
}

.box:hover {
  background-color: lightcoral;
}
```

### 2.4.2 2. 多属性过渡

- 对多个 CSS 属性同时应用过渡效果。
- 示例：

```
.box {
  width: 100px;
  height: 100px;
  background-color: lightblue;
  transition: background-color 0.5s ease, height 0.5s ease;
}

.box:hover {
  background-color: lightcoral;
  height: 150px;
}
```

### 2.4.3 3. 全部属性过渡

- 使用 `all` 关键字对所有可过渡的属性应用过渡效果。
- 示例：

```
.box {
  width: 100px;
  height: 100px;
  background-color: lightblue;
  transform: scale(1);
  transition: all 0.5s ease;
}

.box:hover {
  background-color: lightcoral;
  transform: scale(1.2);
  border-radius: 50%;
}
```

## 2.5 过渡的应用案例

### 2.5.1 案例1：按钮悬停效果

**需求**：创建一个按钮，在鼠标悬停时显示颜色变化和放大效果。

#### HTML 结构

```
<button class="button">点击我</button>
```

#### CSS 样式

```
.button {
  padding: 10px 20px;
  background-color: lightblue;
  border: none;
  color: white;
  font-size: 16px;
  cursor: pointer;
  transition: background-color 0.3s ease, transform 0.3s
ease;
}

.button:hover {
  background-color: lightcoral;
  transform: scale(1.1); /* 放大 1.1 倍 */
}
```

### 2.5.2 案例2：淡入淡出效果

**需求**：创建一个元素，在鼠标悬停时显示淡入淡出效果。

#### HTML 结构

```
<div class="fade-box">淡入淡出</div>
```

#### CSS 样式

```
.fade-box {
  width: 200px;
  height: 100px;
  background-color: lightgreen;
  opacity: 0.5; /* 初始透明度 */
  transition: opacity 1s ease;
}

.fade-box:hover {
  opacity: 1; /* 鼠标悬停时完全不透明 */
}
```

### 2.5.3 案例3：滑动菜单

**需求**：创建一个隐藏的侧边栏，在鼠标悬停时从左侧滑出。

#### HTML 结构

```
<div class="menu">
  <ul>
    <li>首页</li>
    <li>产品</li>
    <li>联系我们</li>
  </ul>
</div>
```

#### CSS 样式

```
.menu {
  width: 300px;
  height: 100vh;
  background-color: rgb(142, 28, 28);
  position: fixed;
  left: -200px; /* 初始状态隐藏 */
  top: 0;
  transition: left 0.5s ease;
}

.menu:hover {
  left: 0; /* 鼠标悬停时滑出 */
}

.menu ul {
```

```
  list-style: none;
  padding: 20px;
}

.menu li {
  margin: 10px 0;
}
```

### 2.5.4 案例4：旋转动画

**需求**：创建一个圆形图标，在鼠标悬停时旋转并改变颜色。

#### HTML 结构

```
<div class="spinner"></div>
```

#### CSS 样式

```
.spinner {
  width: 50px;
  height: 50px;
  background-color: lightblue;
  border-radius: 50%;
  transition: transform 1s ease, background-color 1s ease;
}

.spinner:hover {
  transform: rotate(360deg); /* 旋转 360 度 */
  background-color: lightcoral;
}
```

# 三、2D转换

`transform` 是 CSS 中的一个属性，用于对元素进行 2D 或 3D 转换。以下是常见的 2D 转换方法：

## 3.1 1. 平移（`translate`）

- 将元素沿 X 轴或 Y 轴移动，移动时是根据自身原本位置进行移动
- 示例：

  ```
  transform: translate(50px, 100px); /* 水平移动 50px，垂直移动 100px */
  ```

## 3.2 2. 缩放（`scale`）

- 改变元素的大小。

- 示例：

```
transform: scale(1.5); /* 放大到 1.5 倍 */
transform: scaleX(2); /* 水平方向放大到 2 倍 */
transform: scaleY(0.5); /* 垂直方向缩小到 0.5 倍 */
```

### 3.3 3. 旋转（`rotate`）

- 旋转元素。
- 示例：

```
transform: rotate(45deg); /* 顺时针旋转 45 度 */
```

### 3.4 4. 倾斜（`skew`）

- 倾斜元素。
- 示例：

```
transform: skew(30deg, 20deg); /* 水平倾斜 30 度，垂直倾斜
20 度 */
```

### 3.5 5. 组合转换

- 可以同时应用多个转换效果。
- 示例：

```
transform: translate(50px, 50px) rotate(45deg)
scale(1.2);
```

## 四、练习

悬浮卡牌

篮球